

Assessment 4 Final Project Report

Budgestic – AI-Powered Budgeting & Receipt Analysis App

Subject: IT3090

Assessment: Assessment 4 – Final Demonstration and Project Report

Project Title: Budgestic – AI-Powered Budgeting & Receipt Analysis App

Student Information

Student ID	Student Name
H20937	Rajat Adhikari
ULG20814	Biraj Thapa
ULG20520	Bishwash Thapa

Project Supervisor/Advisor: Dr. Sherif Haggag

GitHub Repository Link

Additional resources such as the Figma application design link, README file, testing results, and documentation for users can be found at the following GitHub repository link:

GitHub Repository: [\[Paste GitHub repository link here before submission\]](#)

1. Project Summary

Budgestic is a budgeting and receipt analysis mobile app that uses artificial intelligence. Budgestic allows users to analyze their personal finance by uploading or scanning their receipts, monitoring their incomes and expenditures, checking cash flow, and even analyzing spending habits using AI technology.

Objective of Project The primary objective of the project is to minimize the manual process of tracking expenditures and enable users to take better decisions regarding their finances. Most users find themselves losing their receipts, forgetting about their expenditures, and even making unnecessary expenditures without budgeting. The solution offered by Budgestic is through a user-friendly mobile application which tracks expenditures.

For Assessment 4, the project provides the entire design of the app, including the navigation process, design layout of the screens, functionality details, testing documentation, and user documentation. The design of the application is done using Figma, while future technical implementation will be done using Flutter, Tesseract OCR, SQLite, and lightweight AI/ML models.

2. Project Objectives

The primary goals of the Budgestic project are as follows:

1. To develop an adaptive and intuitive budgeting app on mobile in Figma.
 2. To enable uploading and scanning of receipts.
 3. To extract details from receipts like item name, price, date, and total amount.
 4. To track the income, expenses, and cash flow.
 5. To offer artificial intelligence-based suggestions on expenditures.
 6. To predict purchases made often.
 7. Navigation flow of the mobile application.
 8. Preparation of all required documents, including final report, testing document, user manual, and README file.
-

3. Scope of the Project

3.1 In Scope

- Mobile application interface design.
- Interactive screen navigation.
- Receipt upload and scanning feature.
- OCR-based receipt data extraction concept.
- Expense tracking.
- Income and cash flow monitoring.
- Spending analysis.
- Basic financial reports and summary.
- Smart recommendation feature.
- Frequently purchased item prediction.
- User documentation and testing documentation.
- README and GitHub supporting files.

3.2 Out of Scope

- Bank API integration.
 - Web-based version of the system.
 - Advanced investment tools.
 - Real-time financial institution syncing.
 - Full commercial deployment.
-

4. Target Users

Target users of Budgestic include:

- Students who require managing their day-to-day expenditure.
 - Workers who wish to track their incomes and expenditures.
 - Mobile phone users who are concerned about managing their personal finance.
 - Mobile phone users who buy groceries regularly and wish to avoid unnecessary purchases.
 - Mobile phone users who desire a budgeting application that is simple and can be used offline.
-

5. Software Development Methodology

The development process was based on the agile model. This model was appropriate for this project since the project involved different phases such as planning, designing the application, testing, and improvement.

Sprint	Focus Area	Main Tasks
Sprint 1	User Interface Planning	Identify required screens and user flow
Sprint 2	App Design	Design welcome, login, dashboard, and navigation screens
Sprint 3	Receipt and Expense Screens	Design receipt upload and expense tracking screens
Sprint 4	Reports and Insights Screens	Design cash flow, reports, and spending insight screens
Sprint 5	Prediction and Notification Screens	Design prediction, reminder, and notification screens
Sprint 6	Testing and Documentation	Test app navigation and prepare final documents

Agile helped the team work in smaller sections, review progress regularly, and make improvements based on feedback.

6. System Requirements

6.1 Functional Requirements

High Priority

- Users can view the app welcome screen.
- Users can navigate to login and create account screens.
- Users can access the dashboard screen.
- Users can access the receipt upload screen.
- Users can use the receipt scanning process.
- Users can access expense tracking screens.
- Users can view cash flow report screens.

Medium Priority

- The system categorises expenses.
- The system analyses spending habits.
- The system generates financial insights.
- The system displays reports and summaries.

Low Priority

- The system predicts frequently purchased items.
 - The system sends notifications for products that may run out.
 - The system provides smart shopping recommendations.
-

6.2 Non-Functional Requirements

Requirement	Description
Usability	The app should be simple and easy to navigate.
Performance	The app should load and move between screens smoothly.
Security	The system should protect user financial data.
Privacy	The system should store user data locally where possible.
Scalability	The design should support future feature expansion.
Accessibility	The interface should use readable text and clear navigation.
Reliability	The system should store expense data accurately.
Maintainability	The design and documentation should be clear for future development.

7. System Design

7.1 Application Architecture

Budgestic operates on the model of a mobile application. This begins with the welcome page, where the user signs in or registers before proceeding to the dashboard page. From this point, the user will be able to utilize functions such as uploading receipts, expense tracking, cash flow analysis, spend analysis, predictions, alerts, and settings.

7.2 System Architecture

The entire process is based on the architecture of the mobile application. The user uploads the receipts via the application and receives financial advice. The OCR component performs the process of extracting relevant information from the receipt. This data is then stored in the local SQLite database. The AI/ML component performs analysis and provides recommendations.

7.3 Data Flow

- Step 1: User activates the Budgestic application on his mobile phone.
- Step 2: User uploads or scans the receipt.
- Step 3: The OCR process starts to read the receipt.
- Step 4: Data cleaning and sorting.
- Step 5: Expenses data stored in SQLite database.
- Step 6: Analysis of income and expenses.
- Step 7: Presentation of results for the user.

7.4 Database Design

The system stores information such as:

- User details.
- Receipt records.
- Expense transactions.
- Income records.
- Categories.
- Spending insights.
- Prediction and notification data.

SQLite is used because it supports local storage, privacy, offline access, and lightweight mobile app development.

8. UI/UX Design

The Budgetic interface was designed in Figma with a mobile-first approach. The app contains screens such as:

- Welcome screen.
- Login screen.
- Create account screen.
- Dashboard.
- Upload receipt screen.
- Expense tracking screen.
- Spending insights screen.
- Prediction screen.
- Cash flow report screen.
- Budget settings screen.
- Notifications screen.
- Profile and settings screen.

Design Principles

- **Simplicity:** The design is simple and easy to understand.
 - **Consistency:** Consistency in button usage, colors, icons, and spacing.
 - **Accessibility:** Texts and labels are easily accessible.
 - **Mobile-first design:** The screens are mobile-first.
 - **Easy navigation:** Easy navigation between sections.
 - **Visual hierarchy:** Financial information is visually prioritized.
 - **Alerts/feedback:** Alerts and status messages are available.
-

9. Implementation Summary

The process of implementing Assessment 4 entailed the design of the whole mobile application design through Figma. This application represents the design, layout, navigation, and user interface design of Budgetic.

Tool / Technology	Purpose
Figma	Mobile app design and UI/UX
GitHub	Repository for documentation and supporting files
Flutter	Mobile application development
Dart	Programming language for Flutter
Tesseract OCR	Receipt text extraction
SQLite	Local database storage
Python	AI/ML model development
Android Studio / VS Code	Development and debugging tools

The app was designed to support budgeting, receipt tracking, spending analysis, and prediction features.

10. Testing Summary

Testing was conducted to check navigation, screen flow, layout clarity, and usability. The main testing areas included:

- App navigation testing.
- Screen layout testing.
- Button and link testing.
- User flow testing.
- Receipt upload screen review.
- Expense tracking screen review.
- Report and insight screen review.
- Usability testing.
- Visual consistency testing.

A separate testing summary document has been prepared and uploaded to GitHub with detailed test cases and results.

11. User Documentation Summary

A separate user documentation file has been prepared for GitHub. It explains how users can open and navigate the Budgetestic application design.

The user documentation includes:

- How to access the app design.
- Login and account creation guide.
- Receipt upload guide.
- Expense tracking guide.
- Viewing reports and insights.
- Using prediction and notification screens.

- Troubleshooting information.
- App limitations and future development notes.

12. Challenges Faced

Challenge	Explanation	Solution
App design limitation	The app is designed in Figma and does not have full backend functions.	Clearly document the current design and future implementation direction.
OCR functionality	Receipt scanning requires OCR integration.	Use Tesseract OCR in the planned technical implementation.
AI prediction	Spending insights and predictions require user data and AI/ML integration.	Use lightweight AI/ML models in future development.
Time management	Multiple screens and documents had to be completed.	Use sprint planning and divide tasks.
Team coordination	Group members needed to contribute regularly.	Use shared tools and GitHub for supporting files.

13. Project Outcomes

Budgetestic project outcomes are:

- Formation of a clearly defined idea of the mobile app.
 - Creation of an entire Figma design of the mobile application.
 - Determination of the main needs of the users.
 - Recording of the functional and non-functional requirements.
 - Creating of UI/UX screens and navigation.
 - Recording of the system architecture and database design.
 - Creation of testing documentation.
 - Creation of user documentation.
 - Creation of README for GitHub.
 - Creation of the final report that covers all aspects of the project development process.
-

14. Future Improvements

Potential future developments for Budgetestic may include:

- Design and develop the application according to Figma design.
- Add an OCR receipt scanner.
- Add an SQLite database.
- Add AI-based spending analysis.
- Predict frequently purchased items.
- Improve user interaction and animations.
- Add bank API integration.

- Add a cloud backup feature.
 - Add report generation in PDF/excel.
 - Add budget goals.
 - Add spending limit alerts.
 - Release the app on Google Play/App store.
-

15. Conclusion

Budgetistic is a mobile application which assists its users in budget management and receipt analysis. This application proves the capability of its users to scan receipts, manage their expenses, cash flow, and receive financial counseling through the medium of a mobile application.

The intended features, user interface, navigation, and technical roadmap of the mobile application are provided here. It would be of immense benefit to students, working individuals, and other general users who want to utilize an easily accessible and privacy-focused budgeting app.

Here are the final report, testing report, user documentation, README file, and project files.

Appendix A: Table of Contributions

Student Name	ID Number	Contribution Percentage
Rajat Adhikari	H20937	33.33%
Biraj Thapa	ULG20814	33.33%
Bishwash Thapa	ULG20520	33.34%
Total		100%

Appendix B: GitHub Contents

The GitHub repository should include:

- Final project report.
- Figma app design link.
- README file.
- Testing summary.
- User documentation.
- Screenshots or UI designs.
- Any supporting diagrams.
- Project files and setup instructions.